

Exact Independent-Set Counts in Motzkin Graphs through Order 144

Carlo Corti
Independent researcher
`carlo.corti@outlook.com`

September 2026

Abstract

We enumerate the independent vertex subsets, including the empty set, of the Motzkin Riordan graph MG_n for every $1 \leq n \leq 144$. A C17 count by vertex deletion agrees at each of these 144 indices with a separately implemented Python count based on independent odd-vertex subsets and their available even vertices. The programs also construct adjacency by different arithmetic: truncated Motzkin-series powers over $\mathbb{F}_2$ and exact integer coefficients from Lagrange inversion, respectively. Relative to the twelve-term table of Cheon and coauthors and the saved OEIS A339592 entry, the locally validated extension comprises indices 13–144, beginning with $a(13) = 104$ and ending with $a(144) = 4839167564472933081443$. We prove the counting identities and a finite 128-bit arithmetic bound, apply a Pell-number upper bound to the Motzkin graph, identify shared assumptions and distinguish the computed range from the uncomputed order 145. The extended b-file, programs and supporting evidence are publicly available with the project repository and its Zenodo archive.

2020 Mathematics Subject Classification: Primary 05C30; secondary 05C69, 05A15.

Key words and phrases: Motzkin graph, Riordan graph, independent set, exact enumeration, integer sequence, computational validation.

1 Introduction

The Motzkin graph MG_n is a labelled Riordan graph on $[n] = \{1, \dots, n\}$. Let $a(n)$ count all its independent vertex subsets, including the empty set. Cheon, Jung, Kang, Kim, Kim, Kitaev and Mojallal gave the values

$$2, 3, 4, 7, 9, 13, 17, 26, 29, 48, 55, 95$$

for $1 \leq n \leq 12$ in their Table 1 [2]. Their concluding section asked for general bounds or exact enumeration for the Motzkin family. N. J. A. Sloane submitted the corresponding OEIS entry A339592 on December 28, 2020; the local snapshot consulted in September 2026 has precisely those twelve terms, offset 1, 1, and the keyword `more` [3]. The underlying Riordan-graph construction and structural framework predate this computation [1].

This article records a finite extension and its local validation. The C17 campaign enumerated each order from 1 through 144. An independently implemented Python algorithm compared every indexed value over the same range. The local b-file was then updated from 1–12 to 1–144; the saved OEIS entry was not edited. No order beyond 144 is admitted from this campaign. The main distinction from the Pascal and Catalan companion graphs is that MG_n is not io-decomposable [1]; their odd-subgraph self-similarity and universal-vertex

consequences do not provide Motzkin values. The even-vertex null subgraph is a previously established Bell-type property [1]. Here it yields a different, weighted enumeration, while the production program uses general vertex deletion. The exact coefficient formulas and an arithmetic bound make both routes explicit.

2 Definition and structural identities

Let $M(z) = \sum_{r \geq 0} M_r z^r$ be the unique formal power series satisfying

$$M(z) = 1 + zM(z) + z^2 M(z)^2, \quad M_0 = 1. \quad (1)$$

The Motzkin graph is the Bell-type Riordan graph $\text{MG}_n = G_n(M, zM)$ [1, 2]. It is simple and undirected, with zero diagonal and, for $1 \leq j < i \leq n$,

$$A_{ij} = A_{ji} = ([z^{i-j-1}]M(z)^j) \pmod{2}. \quad (2)$$

This follows from the Riordan rule $A_{ij} = [z^{i-2}]g(z)f(z)^{j-1} \pmod{2}$ after setting $g = M$ and $f = zM$. The exponent and labels are both one-based: the coefficient degree is $i - j - 1$, not $i - j$. The first m vertices of MG_n induce MG_m for $m \leq n$. We count subsets of this labelled graph, without quotienting by automorphisms. Put $\mathcal{I}(G) = \{S \subseteq V(G) : S \text{ is independent}\}$ and $i(G) = |\mathcal{I}(G)|$; then $a(n) = i(\text{MG}_n)$ and $i(\emptyset) = 1$. For example, $\text{MG}_3 = K_3$, so $a(3) = 4$.

Proposition 2.1. *Every pair of consecutive vertices of MG_n is adjacent, and the even-labelled vertices $E_n = \{2, 4, \dots, 2\lfloor n/2 \rfloor\}$ form an independent set. Consequently, with $F_0 = 0, F_1 = 1$,*

$$2^{\lfloor n/2 \rfloor} \leq a(n) \leq F_{n+2}. \quad (3)$$

The even-vertex assertion is the Bell-type o-decomposition of Cheon et al. [1, Theorem 4.3]; their decomposition also describes the odd-induced graph by a different Riordan pair. The path bound appears in [2, Theorem 3.2 and Corollary 3.3], where the authors use $F(0) = F(1) = 1$, so their $F(n+1)$ is our F_{n+2} . We include the short special-case proof to fix the parity and index conventions used in the computations.

Proof. For $i = j + 1$, the coefficient in (2) is the constant term of M^j , namely 1. If i and j are even, $d = i - j - 1$ is odd and, over $\mathbb{F}_2$, $M(z)^j = (M(z)^{j/2})^2$ has no odd-degree term: squaring a binary power series doubles every exponent. Every subset of E_n is therefore independent, giving the lower bound. Consecutive edges force every independent-set indicator word to avoid consecutive ones; the number of such length- n words is F_{n+2} . $\square$

Cheon et al. used an auxiliary graph Δ_n to bound the counts of io-decomposable Bell-type graphs [2]. The same edge check applies to MG_n without assuming io-decomposability. Its resulting bound is at least as sharp as the path bound and is strict for $n \geq 3$. Let $P_0 = 0$, $P_1 = 1$ and $P_{k+2} = 2P_{k+1} + P_k$ be the Pell numbers. Define

$$\delta_n = \begin{cases} 2P_{(n+1)/2}, & n \text{ odd}, \\ P_{n/2} + P_{n/2+1}, & n \text{ even}. \end{cases}$$

Proposition 2.2. *For every $n \geq 1$,*

$$2^{\lfloor n/2 \rfloor} \leq a(n) \leq \delta_n \leq F_{n+2}.$$

Proof. Let Δ_n have all edges $\{j, j+1\}$ and all $\{j, j+2\}$ with odd j and $j+2 \leq n$. The path edges occur by Proposition 2.1. Since $M_1 = 1$, for odd j we have $[z]M(z)^j = jM_1 \equiv 1 \pmod{2}$; hence the odd-step edges also occur in MG_n . Thus Δ_n is a spanning subgraph of MG_n . Cheon et al. [2, Lemma 3.4] count the independent sets of Δ_n as δ_n with the displayed Pell normalization. Deleting edges cannot decrease the independent-set count, so $a(n) \leq i(\Delta_n) = \delta_n$. The remaining bounds follow from Proposition 2.1 and the fact that Δ_n contains the path on $[n]$. For $n \geq 3$, the path-independent set $\{1, 3\}$ is excluded by Δ_n , so $\delta_n < F_{n+2}$. $\square$

The inclusion is not peculiar to Motzkin graphs: for any proper Bell-type $G_n(g, zg)$ with odd g_1 , its path edges and $[z]g(z)^j = jg_1 \equiv 1 \pmod{2}$ for odd j contain Δ_n . Here we apply that edge argument only to MG_n ; the stronger io-decomposable bounds of [2] are not invoked.

Write $O_n = [n] \setminus E_n$ and let $N(T)$ be the union of the open neighbourhoods of vertices in T . The structural fact in Proposition 2.1 gives the counting identity actually used by the second program.

Proposition 2.3. *For every $n \geq 1$,*

$$a(n) = \sum_{T \in \mathcal{I}(\text{MG}_n[O_n])} 2^{|E_n \setminus N(T)|}. \quad (4)$$

Proof. An independent set S has a unique odd part $T = S \cap O_n$, which must be independent in the odd-induced graph. Once T is fixed, its even part can be any subset of $E_n \setminus N(T)$, because no two even vertices are adjacent. The displayed summands count disjoint classes and include $T = \emptyset$ and the empty set. $\square$

The odd-induced graph is not assumed to be another Motzkin graph. In $\text{MG}_5[\{1, 3, 5\}]$, vertices 1 and 3 are adjacent since $M_1 = 1$, and 3 and 5 are adjacent since $[z]M(z)^3 = 3$; vertices 1 and 5 are nonadjacent since $M_3 = 4$ is even. This induced graph is the path P_3 , whereas $\text{MG}_3 = K_3$, so the two graphs are not isomorphic. This directly illustrates the non-io-decomposability established in [1, Example 4.8]. O-decomposability supplies the even-null fact but does not imply io-decomposability. No universal-vertex formula is used below.

3 Two exact graph and count constructions

3.1 C17 construction and deletion count

All C adjacency arithmetic is in $\mathbb{F}_2$. Writing $b_r = M_r \pmod{2}$, Equation (1) and Frobenius give

$$b_0 = 1, \quad b_r = b_{r-1} + \mathbf{1}_{2|r, r \geq 2} b_{(r-2)/2} \quad (r \geq 1), \quad (5)$$

where addition is modulo 2. The C implementation builds the truncated powers M^j over $\mathbb{F}_2$ by successive convolution with the b_r and applies (2). The graph is held as symmetric adjacency bit masks. Here Frobenius means that, for $B(z) = \sum c_r z^r$ over $\mathbb{F}_2$, $B(z)^2 = \sum c_r z^{2r}$.

For any induced vertex set S and $v \in S$, let $I(S) = i(\text{MG}_n[S])$ denote its independent-set count and $N_S[v]$ its closed neighbourhood within S . The C kernel uses

$$I(\emptyset) = 1, \quad I(S) = I(S \setminus \{v\}) + I(S \setminus N_S[v]). \quad (6)$$

It chooses a vertex of maximum degree in the current induced graph, memoizes counts under the full mask, and handles an edgeless state of size k as 2^k . The two terms partition independent sets according to whether v is excluded or included. Each recursive child has

fewer vertices, proving termination and the exactness of the recurrence; cache collisions replace entries but do not change the mask comparison or the returned count. Counts use checked unsigned 128-bit additions and three 64-bit mask limbs. The campaign source was compiled by its C17 Makefile with GCC 13 at `-O3 -std=c17` and the warning flags `-Wall -Wextra -Wpedantic -Wshadow -Wconversion` and `-Wdouble-promotion -Wformat=2`. The Makefile supplies `SOURCE_SHA` and `MAKE_SHA` from the source and recipe bytes; those macros are required for the source to compile. The executable embeds the resulting identities in its run metadata. Its `run`, `resume` and `verify` modes recover or inspect complete indexed terms, not partial recursion states. The `verify` mode checks state integrity and executable identity; it does not recount the graph.

Proposition 3.1. *For $1 \leq n \leq 192$, the true value $a(n)$ is smaller than 2^{128} . The same bound applies to every induced state counted by the C kernel.*

Proof. Consider six consecutive labelled vertices. Their internal edge coefficients have degree at most 4. Shifting both labels by eight multiplies the relevant power of M by $M^8 = M(z^8)$ over $\mathbb{F}_2$, which is 1 through degree 7. Thus the induced six-vertex graph depends only on its first label modulo 8. Direct application of (5) and (2), followed by exhaustive testing of the 2^6 subsets, gives

first label s	1	2	3	4	5	6	7	8
$i(\text{MG}_{s+5}[\{s, \dots, s+5\}])$	13	14	13	14	15	12	14	15

The C self-test checks these same eight counts; the finite subset calculation is separate from the period-eight proof. Partition [192] into 32 consecutive six-vertex blocks. Restriction of an independent set to the blocks is injective, and each block has at most 15 choices, so $a(192) \leq 15^{32} = (225)^{16} < (256)^{16} = 2^{128}$. As a separate, sharper bound, Proposition 2.2 gives $a(192) \leq \delta_{192} = 6733044458057842709277507685523012161 < 2^{128}$. Since MG_n is an induced prefix, $a(n) \leq a(192)$ for $n \leq 192$. Every independent set of an induced state is also an independent set of MG_n , giving the final assertion. $\square$

This is a numeric representation bound, not a runtime guarantee. The three-mask-limb design accepts orders through 192, but neither it nor the bound supplies a count for 145–192. The implementation rejects arithmetic overflow rather than silently wrapping.

3.2 Python construction and weighted count

The Python checker reconstructs the adjacency independently of the C parity recurrence. Put $w = zM(z)$; Equation (1) becomes $w = z(1 + w + w^2)$. Lagrange inversion [4] gives, for $j \geq 1$ and $d \geq 0$,

$$[z^d]M(z)^j = \frac{j}{d+j} [u^d](1+u+u^2)^{d+j} = \frac{j}{d+j} \sum_{k=0}^{\lfloor d/2 \rfloor} \binom{d+j}{k} \binom{d+j-k}{d-2k}. \quad (7)$$

For $d = 0$ both sides equal 1. For $d > 0$, apply Lagrange inversion to $w^j = z^j M^j$ at degree $d+j$. Python evaluates the integer sum exactly, checks divisibility by $d+j$, and reduces the result modulo 2 only when forming an edge. It then verifies the even-even null condition and enumerates independent odd subsets recursively. At each completed odd subset it adds $2^{|E_n \setminus N(T)|}$, as in (4). The odd recursion either excludes its next vertex or includes it only when no selected odd vertex is adjacent; it therefore visits each valid T once and terminates after $\lceil n/2 \rceil$ decisions.

The programs consequently differ both in coefficient arithmetic and in the counting partition. They still share Equation (2), the one-based vertex convention, and the interpretation of

an independent set. Agreement cannot detect an identical mistake in those shared premises. The original 1–12 table anchors the graph convention externally. Before the official campaign, a disposable direct enumeration tested all 2^n subsets at each $1 \leq n \leq 13$, reproducing that prefix and obtaining 104 at order 13. Its local diagnostic report records this finite check, which shares its coefficient-derived graph with the contemporaneous weighted check and was not an admitted campaign result.

4 Local campaign, validation and provenance

The frozen C17 source, build recipe and executable produced one native run with target 144. Its indexed snapshot has exactly one row for every $n = 1, \dots, 144$; the native log has one start and one commit per order and a completed 144-row terminal record. The Python 3.12 standard-library checker read that frozen snapshot and compared its own count with every indexed row from 1 through 144. Its separate state and log record 144 completed comparisons and zero unchecked rows. The twelve old values match both Table 1 of [2] and the saved OEIS entry [3]. The two exact programs' complete indexed agreement is the basis for the locally validated computational claim; the logs and checksums establish coverage and file identity, rather than mathematical correctness by themselves. The original snapshot, checker state, Makefile and logs are retained with the local project but have not yet been distributed in a public archive. Project source, code-review records and local result evidence underlie this account [5].

Theorem 4.1 (Finite computational result). *With the graph and indexing of (2), the C17 campaign and the distinct Python algorithm give identical exact integers $a(n)$ at each index $1 \leq n \leq 144$. In particular, the 132 locally validated values at indices $13 \leq n \leq 144$ extend the twelve-term reference prefix, and*

$$a(13) = 104, \quad a(144) = 4839167564472933081443.$$

The first index without a result from this campaign is 145.

Evidence and computational justification. Equations (5) and (2) specify the C graph; (6) proves that its terminating count enumerates every independent set once. Equation (7) gives a separate exact edge construction, and Proposition 2.3 proves the Python count. The frozen native snapshot contains consecutive rows 1–144, and the checker state records a matching row at each of those indices. Both logs close their stated ranges without an unchecked index. The b-file comparison also matches those rows by mathematical index, but the later b-file is not a third independent count. These locally audited records supply the finite computational evidence, subject to the shared-definition and implementation limits described above. An integrity check alone would not establish any of the large-order counts. $\square$

n	$a(n)$	Status relative to the saved entry
12	95	Previous table and snapshot
13	104	Locally enumerated and compared
64	4352673893	Locally enumerated and compared
128	18471061891119082367	Locally enumerated and compared
144	4839167564472933081443	Locally enumerated and compared

Table 1: Selected indexed values; the complete local b-file contains 1–144.

The native run took about 13 seconds of wall time and reported a peak resident set of 50,684 KiB; the Python run took about 266 seconds and reported 18,612 KiB. These measurements come from the original run logs. The project's pre-run environment record lists an AMD Ryzen 9 7940HS Linux host and Python 3.12.3, while the Makefile specifies GCC 13; the environment record is context rather than an independent runtime trace. Neither implementation launches parallel workers. These timings are not asymptotic estimates. The final two Python terms each took about 96 seconds in the recorded cumulative run. Earlier isolated diagnostics used a different invocation context; the cause of the timing difference has not been established and has no bearing on the exact-count comparison. The observed timings do not support a projection to order 160 or 192. The campaign did not execute those orders.

The chronology of local data matters. The result-validation report was written when the b-file still held 1–12. A subsequent, separately recorded update copied the complete validated native rows 1–144 into the local b-file, preserving the former twelve-row file as historical evidence. The new b-file has 144 consecutive indexed rows and two terminal line-feed bytes, representing a final blank line. Neither this update nor the computational validation changed the saved OEIS snapshot or constitutes an OEIS submission.

5 Finite structure and limits

The induced-prefix relation yields one general cross-order consequence: $a(n+1) > a(n)$ for every $n \geq 1$, because each independent set on $[n]$ remains one on $[n+1]$ and the singleton $\{n+1\}$ adds a distinct set. It also gives $a(n+1) \leq a(n) + (a(n) - 1) < 2a(n)$: of the old independent sets, at least $\{n\}$ cannot be extended by vertex $n+1$, because consecutive vertices are adjacent. The 144 indexed values satisfy both inequalities. These statements are proved from the graph definition, not conjectured from the finite table.

The even-null identity supplies the lower bound and weighted checker, but does not turn the odd-induced graph into $\text{MG}_{\lceil n/2 \rceil}$. The structural source explicitly excludes Motzkin graphs from the relevant io-decomposable class [1]. The initial Pascal, Motzkin and Catalan rows share a Riordan framework [2], but their defining series and indexed counts differ. Comparing the available finite data did not yield a verified cross-family identity, general recurrence or asymptotic formula for $a(n)$. No such claim is made. In particular, a dyadic universal-vertex identity from another graph family cannot be used to produce $a(145)$ here. Resolving order 145 would require a new, completed enumeration with corresponding validation; additional numeric type capacity alone would not do so.

6 Conclusion

The local computation supplies 132 validated additions to the twelve-term A339592 reference prefix and a complete indexed b-file through 144. The endpoint value is $a(144) = 4839167564472933081443$; order 145 remains unresolved by this campaign. The previously known even-vertex null subgraph enabled a second exact enumeration with coefficient arithmetic and counting logic distinct from the production program. This validates a substantial finite range. Applying the previously used auxiliary-graph method to MG_n yields the general upper bound $a(n) \leq \delta_n$; it does not give an exact formula. General Motzkin-graph enumeration remains open.

Data availability

The C source and build recipe, Python checker, indexed native results, checker state, original logs, pre-run environment record, small-order diagnostic and validation reports, complete 1–144 b-file, and this manuscript’s TeX source and PDF are available in the public project repository <https://github.com/carcorti/A339592>. GitHub releases are archived under the Zenodo concept DOI <https://doi.org/10.5281/zenodo.23010698>. This concept DOI identifies the release family; each deposited version has its own record DOI. The included validation command audits the retained 1–144 results and recomputes small orders, but does not repeat the full campaign. The saved OEIS snapshot still records 1–12, and this work supplies no exact value beyond 144.

AI use disclosure

Large language models were used as research and drafting assistants during the A339592 workflow, including OpenAI Codex and external review systems whose local reports carry the interface or family labels ChatGPT, Claude, DeepSeek, Gemini, GLM, Grok, MiniMax, Muse and Qwen. These labels do not independently establish the serving model, version or backend for every review. No AI system is an author.

Mathematical analysis. AI assistance included literature search, comparison of the Riordan-graph sources and scrutiny of draft arguments. The published Bell-type decomposition and auxiliary-graph counts are credited to their authors. The deductions for MG_n , definitions and finite bounds are stated with proofs or explicit calculations rather than on the authority of a model.

Algorithm and code. The project records identify Codex assistance in the development and hardening of both the C program and the Python checker; they do not establish reliable line-by-line tool authorship. Reviewers supplied code observations and bounded diagnostic programs. A reproduced persistence defect in the earlier C candidate led to the reviewed C source used here. Frozen source and build identities, self-tests, original run records and the Python per-index comparison support the computational claims. The two programs are methodologically distinct but share the graph definition, indexing and a project development process; their agreement is not model-independent proof.

Manuscript. AI assistance was used for drafting, source checks, internal editorial review and the external review reports considered in this revision. Reviewer assertions and submitted diagnostic scripts were weighed against primary sources and local evidence, not accepted as execution records. Final numerical claims rest on indexed artifacts and validation rather than conversational assertions. Carlo Corti bears full scientific responsibility for the mathematical statements, algorithm, implementation, numerical results, validation boundaries and conclusions.

References

- [1] G.-S. Cheon, J.-H. Jung, S. Kitaev and S. A. Mojallal, “Riordan graphs I: Structural properties,” *Linear Algebra and its Applications* **579** (2019), 89–135.
<https://doi.org/10.1016/j.laa.2019.05.033>. Theorem numbering cited here follows the accessible arXiv version, <https://arxiv.org/abs/1710.04604v2>.
- [2] G.-S. Cheon, J.-H. Jung, B. Kang, H. Kim, S.-R. Kim, S. Kitaev and S. A. Mojallal, “Counting independent sets in Riordan graphs,” *Discrete Mathematics* **343** (2020), no. 11, article 112043. <https://doi.org/10.1016/j.disc.2020.112043>. Theorem and lemma numbering cited here follows the accessible arXiv version, <https://arxiv.org/abs/2006.16579v1>.
- [3] N. J. A. Sloane, “A339592: Number of independent sets in Motzkin graph on n nodes,” *The On-Line Encyclopedia of Integer Sequences*, submitted December 28, 2020; local snapshot consulted September 2026. <https://oeis.org/A339592>.
- [4] I. M. Gessel, “Lagrange inversion,” *Journal of Combinatorial Theory, Series A* **144** (2016), 212–249. Journal DOI: <https://doi.org/10.1016/j.jcta.2016.06.018>. Theorem 2.1.1 was checked in the accessible arXiv version, <https://arxiv.org/html/1609.05988v1>.
- [5] C. Corti, *Exact Independent-Set Counts in Motzkin Graphs through Order 144*, GitHub repository and Zenodo archive, 2026. <https://github.com/carcorti/A339592>; <https://doi.org/10.5281/zenodo.23010698>. The Zenodo link is the concept DOI for the project release family, not a version-specific record DOI.